

Chapter 2: User Authentication

Welcome back to the CinéConnect tutorial! In [Chapter 1: Frontend Layout & Navigation](#), we built the sturdy frame of our application, making sure it looks great and is easy to navigate on any device. Now that our "house" has its walls and hallways, it's time to add a crucial feature: a secure way for users to enter and have a personalized experience.

Your Digital ID: Why Authentication Matters

Imagine walking into a private movie screening. You'd need to show your ticket or invitation to prove you're allowed in, right? It's the same for a web application like CinéConnect. "User Authentication" is how we verify who you are, ensure your data is safe, and give you access to personalized features like your favorite movies list or your profile.

Our main goal in this chapter is to understand how CinéConnect lets users securely log in, register, and stay logged in, protecting their personal information and enhancing their experience.

Let's dive into the core concepts that make this secure system work.

The Two Special "Passes": JWT and Refresh Token

When you successfully log into CinéConnect, our backend (the server) gives you two special digital "passes" or "tokens." Think of these like different types of tickets you might use for public transport:

1. **JWT (JSON Web Token) - Your "Daily Ticket":**

- This is a short-term pass, like a single-day bus ticket. It's valid for a short period (e.g., 15 minutes).
- You present this **JWT** every time you want to access something on the backend that requires you to be logged in (like fetching your personalized movie recommendations).
- It proves you're currently logged in and allowed to access features *right now*. If it expires, you can't use it anymore.

2. **Refresh Token - Your "Monthly Pass":**

- This is a longer-term pass, like a monthly subway card. It's valid for a much longer time (e.g., 7 days).
- When your "Daily Ticket" (**JWT**) expires, you don't want to re-enter your password every time. That's where the **Refresh Token** comes in!
- You use your "Monthly Pass" to ask the backend for a brand new "Daily Ticket" without needing to log in again with your email and password. This keeps you logged in smoothly for extended periods.

The "Special Wallet": **httpOnly Cookies**

For enhanced security, where do we keep these "passes"? We store them in something called **httpOnly cookies**.

Imagine you have a very special wallet that only the "ticket master" (our backend server) can open and put passes into. Even if a sneaky script on a website tries to peek into your browser's storage, it cannot access these `httpOnly cookies`. This prevents malicious code from stealing your authentication passes, keeping your session secure.

How You "Log In" to CinéConnect

Let's walk through the most common use case: logging in.

The User Experience

1. You open CinéConnect and go to the login page.
2. You enter your email address and password.
3. You click the "Log In" button.

Behind the Scenes: A Simple Flow

Here's what happens when you click "Log In":

Authentication Flow

1. **User Enters Credentials:** You type your email and password into the login form.
2. **Frontend Sends Request:** The CinéConnect frontend sends this information securely to the backend.
3. **Backend Verifies:** The backend receives your credentials, looks up your user in the `Database`, and checks if the password is correct.
4. **Backend Issues Tokens:** If successful, the backend creates a fresh "Daily Ticket" (JWT) and a new "Monthly Pass" (Refresh Token).
5. **Backend Sets `httpOnly Cookies`:** The backend then sends these two tokens back to your browser, telling it to store them in those special, secure `httpOnly cookies`.
6. **Frontend Updates:** The frontend receives confirmation and knows you're logged in. It updates its internal state to reflect your logged-in status and might display your username or switch to a personalized view.

Using Authentication in the Frontend: The `useAuth` Hook

The good news is that as a frontend developer, you don't need to worry about all the low-level details of creating tokens or handling cookies. CinéConnect provides a super friendly tool called `useAuth` hook.

A "hook" is a special function in React (our frontend framework) that lets you "hook into" React features. The `useAuth` hook gives you:

- `user`: Information about the logged-in user (like their ID and email), or `null` if no one is logged in.
- `isAuthenticated`: A simple `true` or `false` value indicating if a user is logged in.
- `login`, `logout`, `register`: Functions to perform these authentication actions.

Example: Checking if You're Logged In

Remember our `RootLayout` from [Chapter 1: Frontend Layout & Navigation](#)? It used `isAuthenticated` to decide whether to show the `AppShell` (the main frame of the app with navigation).

```
// apps/frontend/src/routes/-RootLayout.tsx (Simplified)
import { Outlet } from "@tanstack/react-router";
import { AppShell } from "@components/layout/AppShell";
import { useAuth } from "@hooks/useAuth"; // Our handy authentication tool!

export function RootLayout() {
  const { isAuthenticated } = useAuth(); // Ask: "Are we logged in?"
  // ... (logic to decide if AppShell is needed)

  if (shouldUseAppShell(pathname, isAuthenticated)) {
    return (
      <AppShell>
        <Outlet />
      </AppShell>
    );
  }

  // If not logged in, or on pages like Login/Register, no AppShell.
  return <Outlet />;
}
```

Here, `useAuth()` gives us `isAuthenticated`, which is then used by `shouldUseAppShell` to make smart decisions about the app's layout. If `isAuthenticated` is `true`, it means a user is logged in, and we can show them the full app experience within the `AppShell`.

Example: Logging In

When you're on the login page, the code to log you in is surprisingly simple thanks to `useAuth`:

```
// apps/frontend/src/pages/Login.tsx (Conceptual)
import { useAuth } from "@hooks/useAuth"; // Our authentication hook
import { useState } from "react";

function LoginPage() {
  const { login } = useAuth(); // We need the 'login' function
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault(); // Stop the browser from refreshing
    try {
      await login({ email, password }); // Call the login function!
      // If successful, the app state will update, and you'll be redirected
      console.log("Login successful!");
    } catch (error) {
      console.error("Login failed:", error);
      // Show an error message to the user
    }
  };
}
```

```

    return (
      <form onSubmit={handleSubmit}>
        <input type="email" value={email} onChange={(e) => setEmail(e.target.value)}
placeholder="Email" />
        <input type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} placeholder="Password" />
        <button type="submit">Log In</button>
      </form>
    );
  }

```

When `login({ email, password })` is called, the `useAuth` hook takes care of sending your credentials to the backend. If the login is successful, your browser gets the special `httpOnly cookies`, and the `useAuth` hook updates its `user` and `isAuthenticated` values. This causes the app to react (e.g., redirect to the home page, as seen in `RootLayout`).

Deeper Dive: How It Works Internally

Let's peek under the hood to see how CinéConnect handles these "passes" and "special wallets."

1. The Frontend Makes the Request (`auth.service.ts`)

When you call `login()` from the `useAuth` hook, it uses a service called `authService` to communicate with the backend.

```

// apps/frontend/src/service/auth.service.ts (Simplified)
// ... imports ...

export const authService = {
  async login(credentials: LoginCredentials): Promise<AuthSessionResponse> {
    const res = await fetch(
      `${resolveApiBaseUrl()}/api/v1/auth/login`, // URL to the backend login
endpoint
    {
      method: "POST",
      credentials: "include", // VERY important: tells browser to send/receive
cookies
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(credentials), // Your email and password
    }
  );

  // If the backend didn't say "OK", something went wrong (e.g., wrong password)
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw { status: res.status, message: "Invalid credentials" };
  }

  // If "OK", parse the response (which only includes public user data)
  const parsed = parseApiEnvelope(await res.json());
  return parsed.data as AuthSessionResponse;
}

```

```

    },
    // ... other methods like register, logout, restoreSession ...
  };

```

The `login` function here uses `fetch` to send a `POST` request to the backend. Notice `credentials: "include"`—this is what tells your browser to automatically send any relevant cookies (like the Refresh Token later on) and accept new ones (like the JWT and Refresh Token upon successful login).

2. The Backend Verifies and Creates Tokens (`auth.service.ts`)

Once the backend receives your login request, its `AuthService` takes over.

```

// apps/backend/src/services/auth.service.ts (Simplified)
import bcrypt from "bcryptjs"; // For secure password comparison
import jwt from "jsonwebtoken"; // For creating JWTs
import { db } from "../db"; // Our database connection
import { users, refreshTokens } from "../db/schema"; // User and Refresh Token tables
import { unauthorized } from "../utils"; // For error handling

// --- Helper Functions (Conceptual) ---
function signAccessToken(userId: number, email: string): string {
  // Creates the short-term JWT (the "daily ticket")
  // It's like stamping your daily ticket with today's date and time.
  return jwt.sign({ userId, email }, getJwtSecret(), { expiresIn: "15m" });
}

async function persistRefreshToken(userId: number): Promise<string> {
  // Creates a unique Refresh Token and saves its secure "hash" in the database.
  // This is like issuing your monthly pass and recording its details in the system.
  const plainToken = "a_very_long_random_string"; // This is actually randomly generated
  const tokenHash = "hashed_version_of_plain_token"; // Stored securely
  await db.insert(refreshTokens).values({ userId, tokenHash, expiresAt: new Date(...) });
  return plainToken; // We return the plain token to be set as a cookie
}

async function buildAuthPayload(userId: number, email: string) {
  const accessToken = signAccessToken(userId, email); // Get the daily ticket
  const refreshToken = await persistRefreshToken(userId); // Get the monthly pass
  return { token: accessToken, refreshToken, userId, email }; // These are packaged
}

// -----

export const AuthService = {
  async login(email: string, password: string) {
    // 1. Find the user in the database by email
    const user = await db.query.users.findFirst({ where: eq(users.email, email)

```

```

});
  if (!user) {
    throw unauthorized("Invalid credentials");
  }

  // 2. Compare the provided password with the securely hashed password in the
  database
  const isPasswordValid = await bcrypt.compare(password, user.password);
  if (!isPasswordValid) {
    throw unauthorized("Invalid credentials");
  }

  // 3. If everything is valid, create and return the access and refresh tokens
  return buildAuthPayload(user.id, user.email);
},
// ... other methods ...
};

```

The `AuthService.login` function does three main things:

1. **Finds User:** It queries the database to find a user with the given email.
2. **Verifies Password:** It uses `bcrypt.compare` to securely check if the provided password matches the stored hashed password.
3. **Builds Auth Payload:** If the password is correct, it calls `buildAuthPayload`. This helper function uses `jwt.sign` to create the **JWT** (Access Token) and `persistRefreshToken` to create and store the **Refresh Token** in the database.

3. The Backend Attaches Tokens as `httpOnly Cookies` (`authCookies.ts`)

After `AuthService.login` returns the tokens, the backend's HTTP response handler (not shown here, but happens after the `AuthService` call) takes these tokens and uses `authCookies.ts` to attach them to the response as `httpOnly cookies`.

```

// apps/backend/src/utils/authCookies.ts (Simplified)
import type { Response } from "express"; // For handling HTTP responses

export const COOKIE_ACCESS = "cc_access"; // Name of the Access Token cookie
export const COOKIE_REFRESH = "cc_refresh"; // Name of the Refresh Token cookie

export function attachAuthCookies(
  res: Response, // The HTTP response object
  accessToken: string,
  refreshTokenPlain: string
): void {
  const baseOptions = {
    httpOnly: true, // Key Security Feature: Makes cookies inaccessible to
    // JavaScript!
    secure: process.env.NODE_ENV === "production", // Only send over HTTPS in
    // production
    sameSite: "lax", // Helps protect against Cross-Site Request Forgery (CSRF)
  };
}

```

```

    path: "/", // These cookies are available for all paths in our application
  });

  // Set the Access Token (daily ticket) cookie with a short expiration
  res.cookie(COOKIE_ACCESS, accessToken, {
    ...baseOptions,
    maxAge: 15 * 60 * 1000, // Expires in 15 minutes (in milliseconds)
  });

  // Set the Refresh Token (monthly pass) cookie with a longer expiration
  res.cookie(COOKIE_REFRESH, refreshTokenPlain, {
    ...baseOptions,
    maxAge: 7 * 24 * 60 * 60 * 1000, // Expires in 7 days (in milliseconds)
  });
}

export function clearAuthCookies(res: Response): void {
  // When you log out, this function tells the browser to delete these cookies.
  res.clearCookie(COOKIE_ACCESS, { path: "/", ...baseOptions });
  res.clearCookie(COOKIE_REFRESH, { path: "/", ...baseOptions });
}

```

The `attachAuthCookies` function is crucial. It sets two cookies, `cc_access` for the JWT and `cc_refresh` for the Refresh Token. The `httpOnly: true` option is the most important part for security. It ensures that these cookies can only be sent to the server and cannot be read or manipulated by any JavaScript code running in your browser, preventing certain types of attacks.

4. The Backend Authenticates Future Requests (`auth.middleware.ts`)

Once you have your "Daily Ticket" (JWT) in an `httpOnly` cookie, how does the backend know you're still logged in when you try to access a protected feature (like your profile)?

This is handled by `authMiddleware`. A "middleware" is like a security checkpoint that every request has to pass through before reaching its destination.

```

// apps/backend/src/middlewares/auth.middleware.ts (Simplified)
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken"; // To verify JWTs
import { getAccessTokenFromRequest } from "../utils/authCookies"; // Helper to
extract cookie

export const authMiddleware = (req: Request, res: Response, next: NextFunction) =>
{
  // 1. Try to get the Access Token (the "daily ticket") from your request
  cookies.
  const token = getAccessTokenFromRequest(req);

  if (!token) {
    // If no token is found, you don't have a ticket, so you're
    "Unauthorized."
    return res.status(401).json({ error: "Unauthorized: No token provided" });
  }
}

```

```

    }

    try {
      // 2. Verify the token: Is it valid? Is it expired? Is it signed by our
      server?
      const decoded = jwt.verify(token, getJwtSecret()) as { userId: number };

      // 3. If valid, the backend now knows who you are! (req.user is set)
      req.user = { userId: decoded.userId };
      next(); // Let the request proceed to its intended destination (e.g.,
      fetch profile data)
    } catch (error) {
      // If verification fails (e.g., token expired or tampered with), you're
      "Unauthorized."
      return res.status(401).json({ error: "Unauthorized: Invalid or expired
      token" });
    }
  };
};

```

The `authMiddleware` intercepts incoming requests. It extracts the `JWT` from your `httpOnly cookie`, verifies its signature and expiration date. If the `JWT` is valid, the backend knows your `userId` and allows your request to proceed. If not, it sends back an "Unauthorized" error.

5. The Frontend Remembers You (But Not the Secrets!) (`auth.store.ts`)

While the actual tokens are safe in `httpOnly cookies` (inaccessible to JavaScript), the frontend still needs to know *who* is logged in to display personalized information, like your email in the navigation. This is where `useAuthStore` comes in.

```

// apps/frontend/src/stores/auth.store.ts (Simplified)
import { create } from "zustand"; // Our state management library
import { persist } from "zustand/middleware"; // To save state across browser
restarts

export interface AuthUser {
  userId: number;
  email: string;
}

interface AuthState {
  user: AuthUser | null;
  setUser: (user: AuthUser) => void;
  clearAuth: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      user: null, // Initially, no user is logged in
      setUser: (user) => {
        set({ user }); // When login is successful, store public user data

```



```
    },
    clearAuth: () => {
      set({ user: null }); // When logging out, clear user data
    },
  })),
  {
    name: "cineconnect-auth", // Name for local storage
    partialize: (state) => ({ user: state.user }), // Only persist the 'user'
  },
  field
}
)
);
```

`useAuthStore` is a small piece of "global state" that holds non-sensitive information about the logged-in user, like their `userId` and `email`. This information is then used to update the UI. Crucially, it does *not* store the actual `JWT` or `Refresh Token` – those remain securely in the `httpOnly cookies`.

JWT vs. Refresh Token: A Quick Comparison

Here's a summary of the differences between our two "passes":

Feature	JWT (Access Token)	Refresh Token
Purpose	Access protected resources	Get new Access Tokens
Lifespan	Short (e.g., 15 minutes)	Long (e.g., 7 days)
Storage	<code>httpOnly cookie (cc_access)</code>	<code>httpOnly cookie (cc_refresh)</code>
Client Access	No (due to <code>httpOnly</code>)	No (due to <code>httpOnly</code>)
Backend Storage	No (stateless, verified by signature)	Yes (stored securely in database)
Analogy	Daily ticket for immediate use	Monthly pass to renew daily tickets

Conclusion

In this chapter, we've unravelled the essential concept of user authentication in Cin Connect. We learned about the two special "passes"—the short-term `JWT` (Access Token) for immediate access and the long-term `Refresh Token` for seamless re-authentication—and how both are securely stored in `httpOnly cookies`. We saw how the `useAuth` hook simplifies login and registration for the frontend, and we peered behind the curtain to understand how the backend issues, stores, and verifies these tokens to keep your experience secure and personalized.

Now that we know how users log in and out, in the next chapter, we'll explore how Cin Connect handles and displays all the dynamic information you see on your screen, like movie lists and user details, in [Frontend Data & State Management](#).